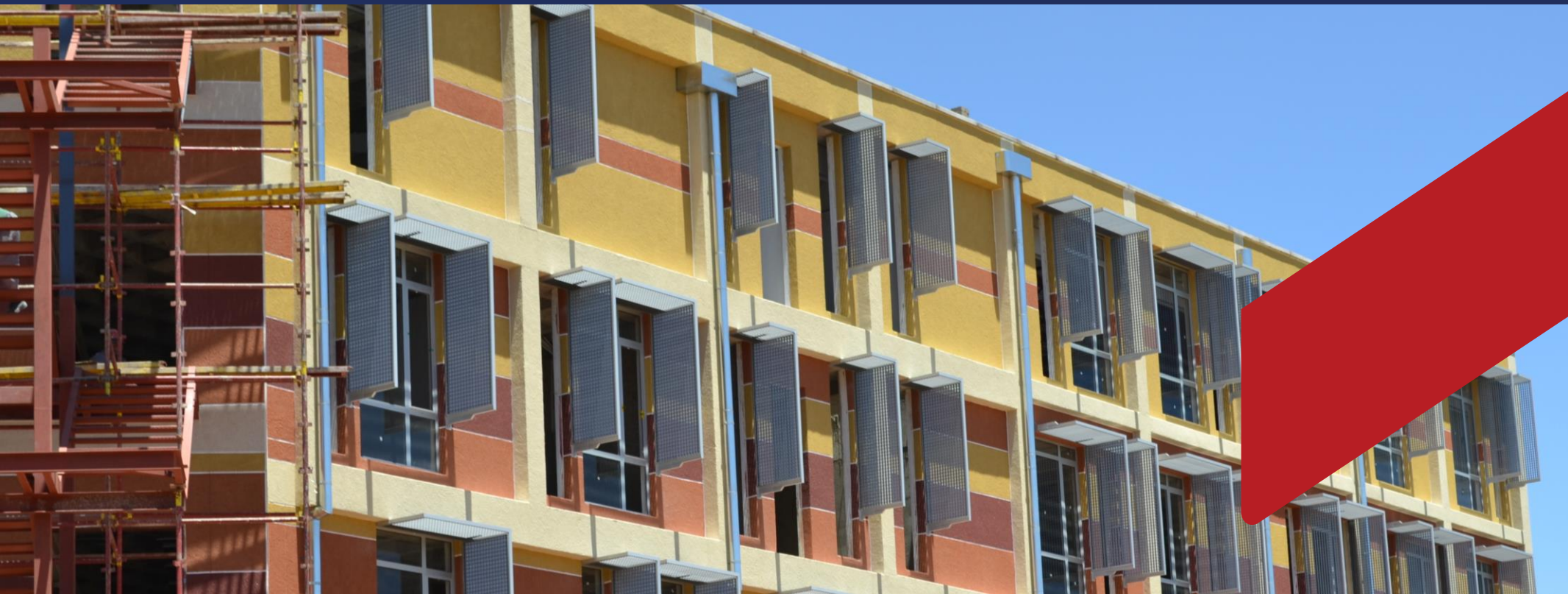
NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Lesson 4

An Exciting Career in Entomology Awaits You: A Bug-Tracking Database



NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Overview

- Analysing failure reports, Debugging/fault isolation techniques, Defect analysis
- Software reliability and Static Analysis of Software Systems

Why Bother? The Case for a Formal Bug-Tracking System

- By *defect* or *bug*, it mean some problem present in the system under test that would cause it to fail to meet a customer's or user's reasonable expectations of quality.
- A bug is a potential source of product dissatisfaction.
- A *bug report* is a technical document that describes the various symptoms or failure modes associated with a single bug.
- A good bug report provides the project management team the information they need to decide when and whether to fix a problem.
- A good bug report also captures the information a programmer will need to fix and debug the problem.
- Because a bug report is so specific and discrete, it is the most tangible product of testing and represents a well-defined opportunity for the project team to make a decision to increase the quality of the system.
- Because we want to focus on problems in the system under test, not in the development or maintenance process, testers should not file bug reports against process breakdowns such as late delivery of test releases.
- There is also the possibility that not everything that at first appears to be a bug symptom is determined, ultimately, to arise from an actual problem in the system under test.

Why Bother? The Case for a Formal Bug-Tracking System

- When the expected results and the actual results of a test differ, some people refer to that situation as an *anomaly*.
- These same people refer to the observation of an anomaly as an *incident*.
- They distinguish an incident from an actual bug-related failure, and prefer to talk about incident
- reports rather than bug reports.
- The phrase *bug report* is common and harmless, but you should keep in mind that false positives can
- and do happen.
- This issue has implications beyond merely complicating the process of tracking bugs.
- A *bug-tracking system* is some program or application that allows the project team to report, manage, and analyze bug reports and bug trends.
- Functionally, most bug-tracking systems provide a form that allows us to report and manage specific bugs, a set of stored reports and graphs that allow us to analyze, manipulate, and output this bug data in various ways, and a customizable workflow or life cycle that provides for orderly bug

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Systematic approach to bug tracking provides some important benefits

Faculty of Computing and Informatics

- A bug-tracking database facilitates clear communication about defects. Well-written, standardized reports tell the story much better than free-form emails or shouts across a hallway or to the next cubicle.
- Using a database allows automatic, sequential bug numbering (a useful way to keep track of and refer to bugs), and it provides a number of analysis and reporting options that are not available with a flat file.
- If you have never looked into the various types of product and process metrics that can be derived from defect information, be ready to be surprised.
- A bug-tracking database allows the development team to fix problems based on importance to the project team, project management, the customers, and the users.
- With a more informal approach, the tester who has the best rapport with the most developers is often the one whose bugs get fixed, which is hardly a good strategy for focusing scarce project resources on the most important bugs. You can manage bugs throughout their life cycle, from the initial report to the final resolution.
- This ensures that bugs don’t fall through the cracks, and it keeps attention focused on the important bugs that need to be fixed as quickly as possible.
- As bugs progress through their life cycle, developers, testers, and managers learn new information. A well-designed bug-tracking database allows you to capture this history and then refer to it later when you are looking at the status of the bugs. Every bug report that is closed in your database is a defect that might otherwise have gone into production, causing support calls, bad reviews, and lost sales. Every bug report that is not closed when the product ships provides technical support with useful advance warning and proves that your testers found the bug in the event that it crops up in the field.

So, What Seems to Be the Problem? The Failure Description

- Bug-tracking systems allow you to capture a lot of information about each bug.
- Some of this information is classifying data, generally selected from some set of values.
- For example, severity and priority ratings for bugs often run from 1 (most serious) to 5 (least serious).
- Other information is more narrative, describing what happens when the bug symptoms appear, how the bug was fixed, and so forth.
- So possible pieces of information in detail, but let's start with the narrative data that captures the account of the problem as told by the report's author.
- This is the heart of any bug-tracking and reporting system, the *failure description*.
- The failure description is the message of the bug report.
- The failure description is the tester's first and best opportunity to communicate clearly with the programmers and the project team about a problem.
- Done properly, the failure description captures in simple prose the essentials of the bug.
- Done poorly, the failure description obfuscates the bug and misleads the reader.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

What does a good failure description look like? - A good SpeedyWriter bug report

- 1. Started SpeedyWriter editor, then created new file.
- 2. Typed four lines of text, repeating "The quick fox jumps over the lazy brown dog" each time.
- 3. Highlighted all four lines of text, then pulled down the font menu, and selected Arial.
- 4. All text converted to control characters, numbers, and other apparently random binary data.
- 5. Reproduced three out of three tries.
- Isolation
- New to build 1.1.018; same test case passed against builds 1.1.007 (System Test entry) through 1.1.017.
- Reproduced with same steps using Wingdings and Symbol fonts.
- On vague suspicion this was a formatting problem, saved file, closed SpeedyWriter and reopened file, but data corruption remained.
- Saving file before changing font prevents bug.
- Bug does not occur with existing files.
- Only happens under Windows XP, not Solaris, Mac, or other Windows flavors.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Ten Steps to Better Bug Reports

1. **Structure:** Test thoughtfully and carefully, whether you're using reactive techniques, following scripted manual tests, or running automated tests.
2. **Reproduce:** My usual rule of thumb is to try to reproduce the failure three times. If the problem is intermittent, report the rate of occurrence; for example, one in three tries, two in three tries, and so forth.
3. **Isolate:** See if you can identify variables— for example, configuration changes, workflow, data sets— that might change the symptoms of the bug.
4. **Generalize:** Look for places that the bug's symptoms might occur in other parts of the system, using different data, and so forth, especially where more severe symptoms might exist.
5. **Compare:** Review the results of running similar tests, especially if you're repeating a test run previously.
6. **Summarize:** Write a short sentence that relates the symptom observed to the customers' or users' experiences of quality, keeping in mind that in many bug review or triage meetings, the summary is the only part of the bug report that is read.
7. **Condense:** Trim any unnecessary information, especially extraneous test steps.
8. **Be clear:** Use clear words, avoiding especially words that have multiple distinct or contradictory meanings; for example, ''The ship had a bow on its bow,'' and ''Proper oversight prevents oversights,'' respectively.
9. **Neutralize:** Express yourself impartially, making statements of fact about the bug and its symptoms and avoiding hyperbole, humor, or sarcasm. Remember, you never know who'll end up reading your bug report.
10. **Review:** Have at least one peer, ideally an experienced test engineer or the test manager, read the bug report before you submit it.

Flexible Reporting: Beginning to Construct a Database

- You could implement every piece of advice in the previous section in a bug-reporting system that relies on email or a flat file.
- Some projects use spreadsheets or word processors to track bugs, but the reporting options were sparse, limited to little more than a printout of the worksheet or document. (Spreadsheets can summarize and graph data, but they have poor text-reporting capabilities. Word processors have good text formatting, but poor analysis and graphical abilities.)
- To store, manipulate, search, analyze, and report large volumes of data flexibly, you need a database.
- There are certainly many options for commercial and freeware bug-tracking databases.
- However, many companies do build their own.
- You can use Microsoft Access for the database shown here, but you can use StarOffice, FileMaker, Oracle, or any other relational-database application.
- You should look for in a commercial or freeware bug-tracking system if you decide to acquire one.
- Minimally, a bug-tracking database stores narrative information about the failure (e.g., the summary, steps to reproduce, and isolation) together with classifying information such as a sequential ID number, the project name, the author of the report, and the date the report was filed.

NAMIBIA UNIVERSITY OF SCIENCE AND TECHNOLOGY

The design for a basic bug-tracking database

Faculty of Computing and Informatics

Field Name	Data Type	Description
Bug ID	Auto Number	The bug identifier.
Project Name	Text	The Project against which the bug appeared.
Tester	Text	The tester's name.
Date Opened	Date/Time	The date the report was opened.
Summary	Text	The bug report summary.
Steps to Reproduce	Memo	How the tester was able to reproduce the bug.
Isolation	Memo	what the tester did to determine the bug was real, and to find influencing factors.

Field Properties

General | Lookup

Field Size	Long Integer
New Values	Increment
Format	
Caption	
Indexed	Yes (No Duplicates)
Smart Tags	
Text Align	General

A field name can be up to 64 characters long, including spaces. Press F1 for help on field names.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

A bug detail report

Faculty of Computing and Informatics

Bug ID	Project Name	Tester	Date Opened
1	SpeedyWriter I.0	Bob Chien	7/22/2009

Summary

Arial, Wingdings, and Symbol fonts corrupt new files.

Steps to Reproduce

1. Started SpeedyWriter editor, then created new file.
2. Typed four lines of text, repeating "The quick fox jumps over the lazy brown dog" each time.
3. Highlighted all four lines of text, then pulled down the font menu, and selected Arial.
4. All text converted to control characters, numbers, and other apparently random binary data.
5. Reproduced three cut of three tries.

Isolation

New to build 1.1.018; same text case passed against builds 1.1.007 (System Test entry) through 1.1.017.
Reproduced with same steps using Wingdings and Symbol fonts.
On vague suspicion this was a formatting problem, saved file, closed SpeedyWriter and reopened file, but data corruption remained.
Saving file before changing font prevents bug.
Bug does not occur with existing files.
Only happens under Windows XP. not Solaris. Mac. or other Windows flavors.

Putting the Tracking in Bug Tracking: Adding Dynamic Information

- So far, you have a database that is fine for *reporting* bugs but not for *tracking* them.
- It still lacks a way to include dynamic information, a mechanism to trace the steps through which a bug must move on its way to resolution.
- Having this information available would allow you to answer questions such as these: ***Who is currently responsible for pushing the bug toward closure? Is the resolution process stuck or moving forward smoothly? When will the bug be fixed?***

Using States to Manage Bug Life Cycles

- The aim of reporting problems is to bring them to the attention of the appropriate people, who will then cause the most important bugs to be fixed— or will at least attempt to have them fixed.
- The test organization then must either confirm or rebut the proposed fixes.
- Other possible outcomes for a particular bug report include its cancellation as a non-problem, its closure as a duplicate, or its indefinite deferral because of a lack of interest.
- In other words, a bug report should go through an identifiable life cycle, with clear ownership at each phase or state in its life cycle.

NAMIBIA UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

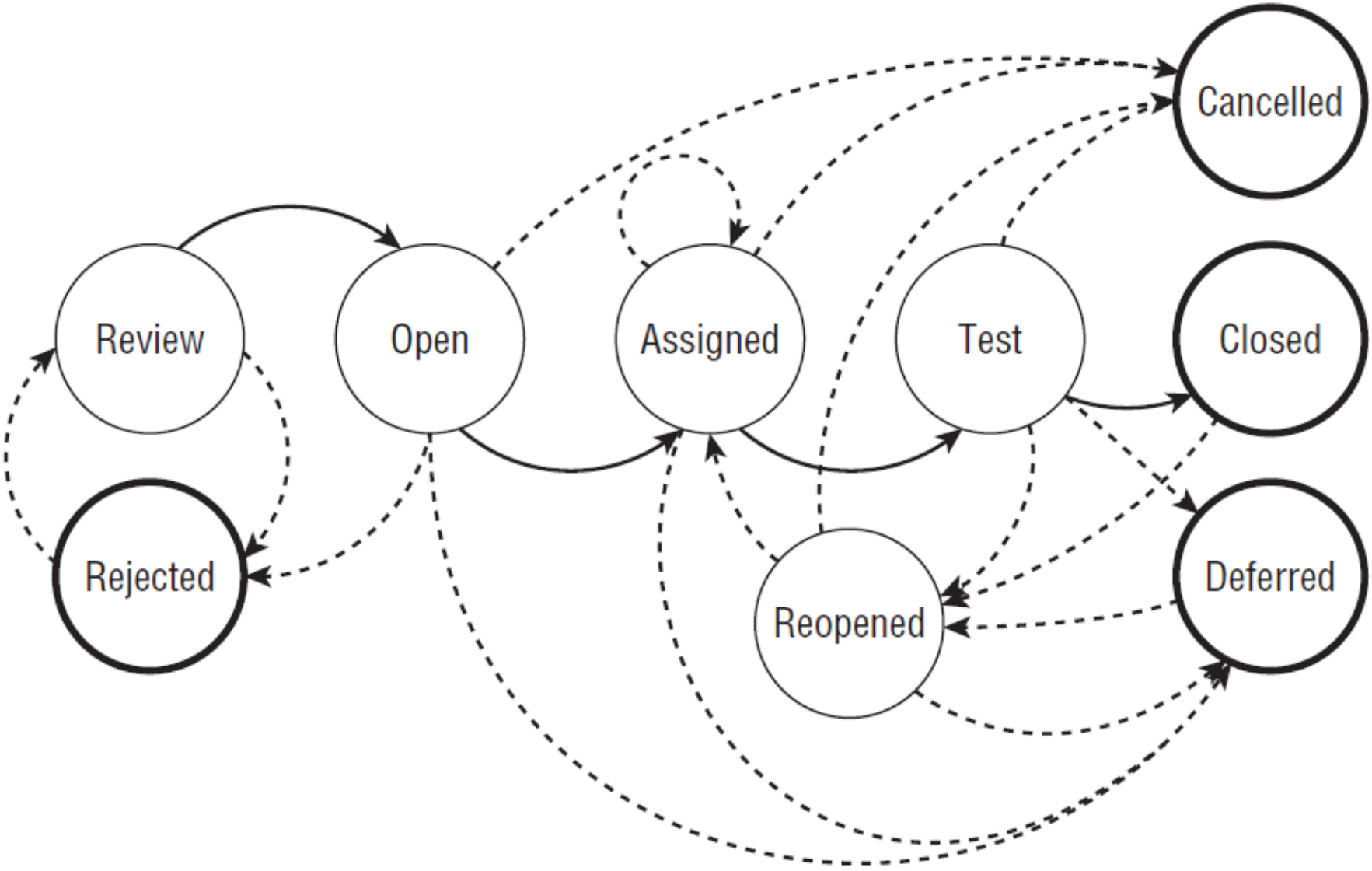
One life cycle used: Example

Review. When a tester enters a new bug report in the bug-tracking database, the bug-tracking database holds it for review before it becomes visible outside the test team. If non-testers can report bugs directly into the system, then the managers of those non-testers should determine the review process for those non-tester bug reports. **Rejected**. If the reviewer decides that a report needs significant rework— either more research and information or improved wording — the reviewer rejects the report. This effectively sends the report back to the tester, who can then submit a revised report for another review. The appropriate project team members can also reject a bug report after approval by the reviewer. **Open**. If the tester has fully characterized and isolated the problem, the reviewer opens the report, making it visible to the world as a known bug. **Assigned**. The appropriate project team members assign it to the appropriate development manager, who in turn assigns the bug to a particular developer for repair. **Test**. Once development provides a fix for the problem, it enters a test state. The bug fix comes to the test organization for confirmation testing (which ensures that the proposed fix completely resolves the bug as reported) and regression testing (which addresses the question of whether the fix has introduced new problems as a side effect). **Reopened**. If the fix fails confirmation testing, the tester reopens the bug report. If the fix passes confirmation testing but fails regression testing, the tester opens a new bug report. **Closed**. If the fix passes confirmation testing, the tester closes the bug report. **Deferred**. If appropriate project team members decide that the problem is real but choose either to assign a low priority to the bug or to schedule the fix for a subsequent release, the bug report is deferred. Note that the project team can defer a bug at any point in its life cycle. **Cancelled**. If appropriate project team members decide that the problem is not real, but rather is a false positive, the bug report is cancelled.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

A bug report life cycle or workflow



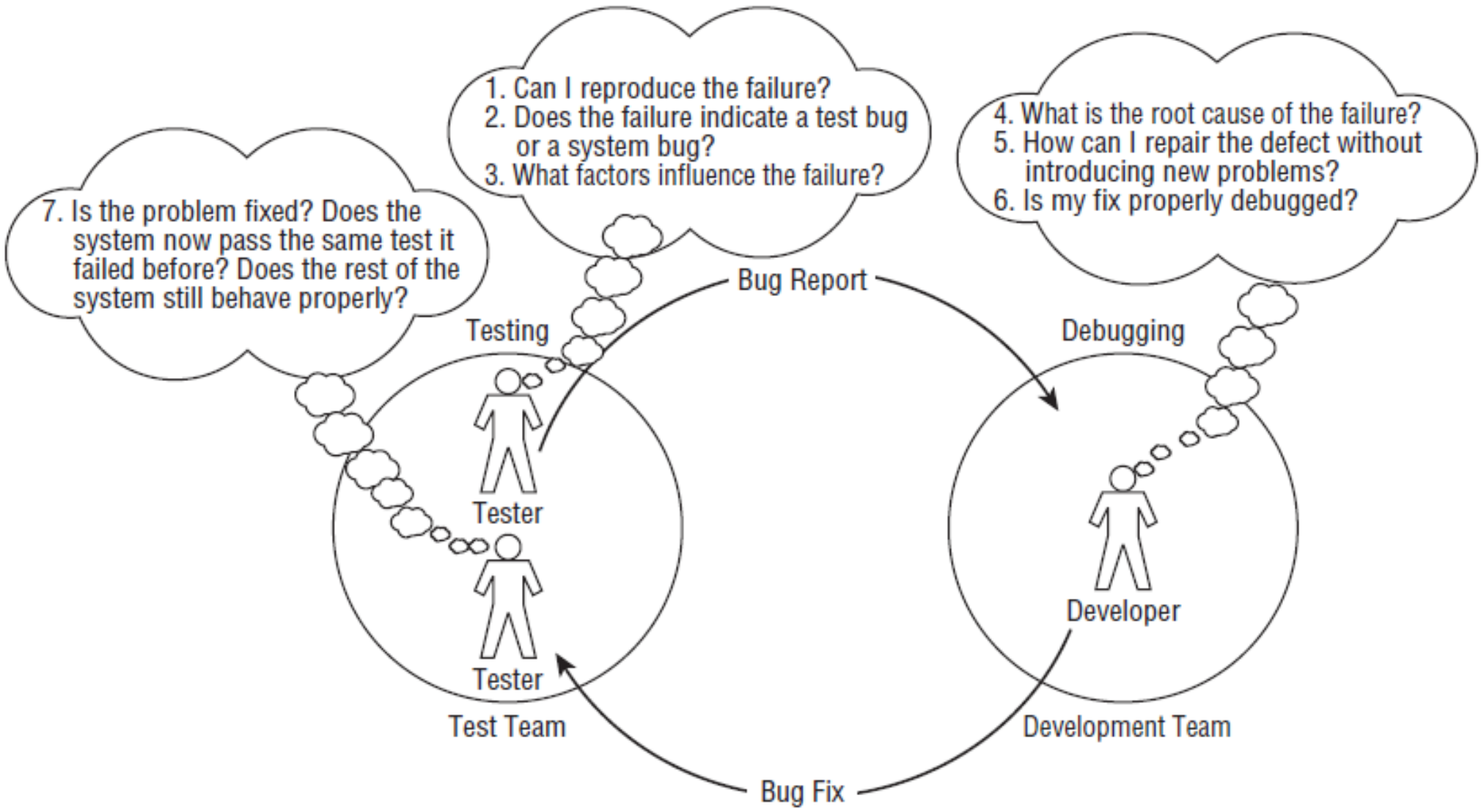
Along this life cycle, the participants must answer a series of questions:

1. What is the exact and minimal sequence of steps required to reproduce the symptoms of the bug? How often do these steps successfully reproduce it?
2. Does the failure indicate a test bug or a system bug? In other words, does the anomalous result originate from a test artifact or a tester error, or from system misbehavior that could affect customers?
3. What external factors influence the symptoms? In the SpeedyWriter example, one way the tester looked for influential factors was by trying different fonts.
4. What is the root cause of the problem, in the code, the electronics, the network, or the environment? Root causes are internal factors.
5. How can the problem be repaired without introducing new problems?
6. Are the changes properly debugged?
7. Is the problem fixed? Does it now pass the same test it failed before, and does the rest of the system still behave properly?

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Questions, players, and handoffs in the bug life cycle



NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

A bug entry form with dynamic information

Faculty of Computing and Informatics

Bug ID	Project Name	Tester	Date Opened	Severity	Priority	RPN
1	SpeedyWriter 1.0	Bob Chien	7/22/2009	1	2	2

State	Owner	Estimated Fix
Assigned	Muhammad Zamanzadeh	8/12/2009

Summary

SpeedyWriter for Windows XP scribbles on the file under edit if Arial font is selected.

Steps to Reproduce

1. Open SpeedyWriter and create a new file.
2. Type in two or more lines of random text. (It doesn't matter what.)
3. Highlight the text, pull down the font menu, and select Arial.
4. Text is transformed into meaningless garbage.
5. I was able to reproduce this problem three out of three tries.

Isolation

On the vague suspicion that this was just a formatting problem, I saved the file, closed SpeedyWriter and reopened the file. The garbage remained.
If you save the file before Arializing the contents, the bug does not occur.
The bug does not occur with existing files.
This only happens under Windows XP
This bug doesn't recur with other fonts.

Status

7/23: Reviewed and opened this issue.
7/29: Assigned to Muhammad, who thinks he can fix it in a couple weeks.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

A bug detail report with dynamic information

SpeedyWriter Bug Detail

Bug ID	Project Name	Tester	State	Date Opened
1	SpeedyWriter I.0	Bob Chien	Assigned	7/22/2009

Severity	Priority	Risk Priority Number	Owner	Estimated Fix
1	2	2	Muhammad Zamanzadeh	8/12/2009

Summary

SpeedyWriter for Windows XP scribbles on the file under edit if Arial font is selected.

Steps to Reproduce

1. Open SpeedyWriter and create a new file.
2. Type in two or mere lines of random text. (It doesn't matter what)
3. Highlight the text, pull down the font menu, and select Arial.
4. Text is transformed into meaningless garbage.
5. I was a able reproduce this problem three out of three tries.

Isolation

On the vague suspicion that this was just a formatting problem. I saved the file, closed SpeedyWriter and reopened the file. The garbage remained.
If you save the file before Arializing the contents, the bug does net occur.
The bug does not occur with existing files.
This only happens under WindowsXP
This bug doesn't recur with other fonts.

Status

7/23: Reviewed and opened this issue.
7/29: Assigned to Muhammad. who thinks he can fix it in a couple weeks.

Finishing Touches: Capturing Bug Data for Analysis

- You now have a fully adequate bug-tracking database that you can use to manage bugs through their entire life cycle, to produce summary reports for management and detail reports for developers, and to keep track of who is responsible for moving bugs forward. You might want to design additional reports or make some cosmetic changes, but the database is essentially complete.

Where the Bug Came From: Resolution and Root Cause

- The simplest way to keep track of where bugs come from is to ask the programmers and other engineers who fix the bugs to take a note.
- By adding a Resolution field, this note can be entered to describe how the problem was solved. (Alternatively, you could use the Status field for this purpose, but a
- separate field is more useful for reports.)
- Through database programming, you can force completion of this field.
- Any time a bug is closed, some notation should be made.
- The problem with free-form resolution fields is that it's hard to do analysis on them. In addition, if people forget to write anything down— or write something so cryptic that no one but the engineer who wrote it can figure it out — then you might be missing data for many bugs.
- The solution to this problem is for the project team— specifically, the development or project
- manager — to institute a root cause analysis process and a bug taxonomy.
- If you are the first test manager your company has ever had, with no exposure to root cause analysis, and your development process consists of hacking and flailing away until the product appears to work, root cause analysis is probably beyond the level of process maturity at your company.

For software, you might consider the following bug taxonomy

- ***Functional***

Specification. The specification is wrong.
Function. The specification is right, but implementation is wrong.
Test. The system works correctly, but the test reports a spurious error.

- ***System***

Internal Interface. The internal system communication failed.
Hardware Devices. The hardware failed.
Operating System. The operating system failed.
Software Architecture. A fundamental design assumption proved invalid.
Resource Management. The design assumptions are OK, but some implementation of the assumption is wrong.

- ***Process***

Arithmetic. The program incorrectly adds, divides, multiplies, factors, integrates numerically, or otherwise fails to perform an arithmetic operation properly.
Initialization. An operation fails on its first use.
Control or Sequence. An action occurs at the wrong time or for the wrong reason.
Static Logic. Boundaries are misdefined, logic is invalid, ‘‘can’t happen’’ events do happen, ‘‘won’t matter’’ events do matter, and so forth.
Other. A control-flow or processing error occurs that doesn’t fit in the preceding buckets.

NAMIBIA UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

For software, you might consider the following bug taxonomy

- ***Data***

Type. An integer should be a floating point, an unsigned integer stores or retrieves a negative value, an object is improperly defined, and so forth.

Structure. A complex data structure is invalid or inappropriate.

Initial Value. A data element's initialized value is incorrect. (This might not result in a process initialization error.)

Other. A data-related error occurs that doesn't fit in the preceding buckets.

- ***Code***

A typo, misspelling, stylistic error, or other coding error occurs that results in a failure.

- ***Documentation***

The documentation says the system does *X* on condition *Y*, but the system does *Z* — a valid and correct action —instead.

- ***Standards***

The system fails to meet industry or vendor standards, follow code standards, adhere to naming conventions, and so forth.

- ***Other***

The root cause is known, but fits none of the preceding categories.

- ***Duplicate***

Two bug reports describe the same bug. (This can happen when two testers report the same symptom, or when two testers report different symptoms that share the same underlying code problem.).

For software, you might consider the following bug taxonomy

- ***NAP***

The behavior described in the bug report is not a problem. The report arises from a misunderstanding on the part of the tester about correct behavior. This situation is distinct from a test failure (whose root cause is categorized as *functional/test*) in that this is *tester* failure.

- ***Bad Unit***

The bug is a real problem, but it arises from a random hardware failure that is unlikely to occur in the field. (If the bug indicates a lack of reliability in some hardware component, this is not the root cause.)

- ***RCN***

A root cause is needed; the bug is confirmed as closed by test, but no one in development has supplied a root cause.

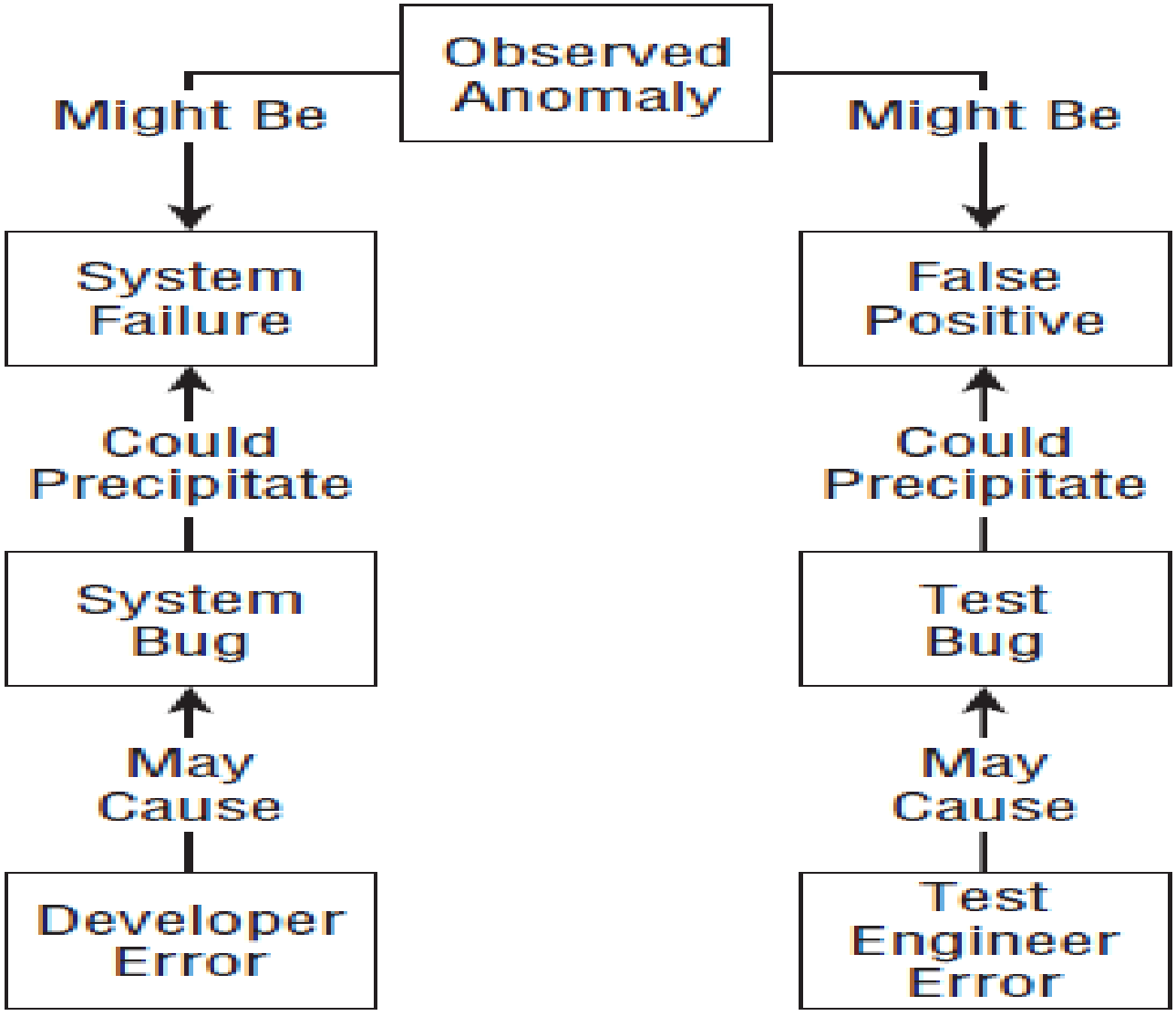
- ***Unknown***

No one knows what is broken. This root cause usually fits best when an intermittent failure doesn't appear for quite awhile, leading people to conclude that some other change fixed the bug as a side effect. Why go to the trouble of capturing this information? First, gathering root cause data allows you to apply the Pareto principle— focus on the vital few, not the trivial many — when the time comes to try to improve your development process. For example, if most DataRocket failures result from bad components, you should revisit your choice of suppliers. Next, if you use industry-standard bug taxonomies, you can compare root cause statistics for your projects against the averages. You can also accumulate historical data over the course of your projects that you can use for in-house comparisons.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Bugs and their root causes



Managing Bug Tracking

- When projects are in a test execution phase, you will probably spend 10 to 20 percent of your time working with bugs, either those already reported or those about to be reported.
- Because testing should mitigate quality risks, and because a bug report documents the transition of a quality risk into a known bug, we consider bug reports one of the most important products of the test effort.
- Time devoted to understanding and managing bug data is time well spent.
- The preceding section on metrics might have given you the impression that managing bugs consists mostly of analyzing data and gleaning meaning.
- You'll soon discover two other areas of concern.

The first is staying aware of the politics and potential misuse of the bug data.
The second is dealing with certain types of problematic bug reports that can consume a disproportionate share of your time.

Don’t Fail to Build Trust

- Most experienced developers support bug-tracking efforts.
- Nevertheless, some individuals—especially those who have experienced highly adversarial test/development relationships— won’t disclose any more information to you than required, and they might do so with an uncooperative attitude.

For example, someone once worked with a development manager who wanted to argue over every bug report. Bug triage meetings with this guy were a misery of semantic arguments punctuated with repeated assertions that any bug he had succeeded in ignoring for the past three or four weeks had probably been fixed in some recent release and should be retested by my team. In his mind, our relationship was by nature purely adversarial. There was no trust.

- Some situations are irretrievable.
- Developers who are convinced that a written bug report is one step removed from a written warning in their personnel files probably will never trust you.
- Most developers, though, approach testing with an open mind.
- They understand that testing can provide a useful service to them in helping them fix bugs and deliver a better product.
- *How do you keep the trust and support of these developers?*

How do you keep the trust and support of these developers?

- Don't take bugs personally, and don't become emotional about them.
- The times you anoint yourself the defender of quality and act holier-than-thou with your colleagues are moments in your professional life that you look back on with embarrassment and regret.
- Even if you're right on the issue, you're doing the wrong thing because you end up alienating people with whom you need to maintain a professional and trusting relationship.
- Submit only quality bug reports: a succinct summary, clear steps to reproduce, evidence of significant isolation work, accuracy in classification information, and a conservative estimate in terms of priority and severity.
- Also try to avoid cheap shot bug reports that can seem like carping.
- Be willing to discuss bug reports with an open mind.
- No matter how convinced you are that a program or a computer is malfunctioning, you should listen to developers when they explain that the bug is actually correct behavior. If you make the extra effort to discuss their opinions, it goes a long way toward keeping the flow of information moving.
- If developers want you to change something in your bug-reporting process, be open to their suggestions.
- They are, after all, some of the key customers for your bug report product.
- If they want reports delivered in a different fashion, advance warning before summaries go to management, or additional information in the reports, try to accommodate them whenever it's practical to do so.

Don’t Be a Backseat Driver

- The test manager needs to ensure that testers identify, reproduce, and isolate bugs.
- It’s also part of the job to track the bugs to conclusion and to deliver crisp bug status summaries to senior and executive management.
- These roles differ, though, from managing bug *fixes*.
- In most of the organizations, managing the bug fix process was the development manager’s job; ensuring that this process moves at a satisfactory pace was the program manager’s job.
- Even if some managers encourage you, don’t get sucked into doing either.
- If you, as an outsider, make it your job to nag developers about when a specific bug will be fixed or to pester the development manager about how slow the bug fix process is, you are setting yourself up for a highly antagonistic situation.
- Reporting, tracking, re-testing, and summarizing bugs are your worries.
- Whether any particular bug gets fixed, how it gets fixed, and when it gets fixed are someone else’s concerns.

Don’t Make Individuals Look Bad

- It is a bad idea to create and distribute reports that make individuals look bad.
- There’s probably no faster way to guarantee that you will have trouble getting estimated fix dates out of people than to produce a report that points out every failure to meet such estimated dates.
- Creating reports that show how many bug fixes resulted in reopened rather than closed bugs, grouped and totaled by developer, is another express lane to bad relationships.
- Again, managing the developers is the development manager’s job, not yours.
- No matter how useful a particular report seems, make sure that it doesn’t bash individuals.

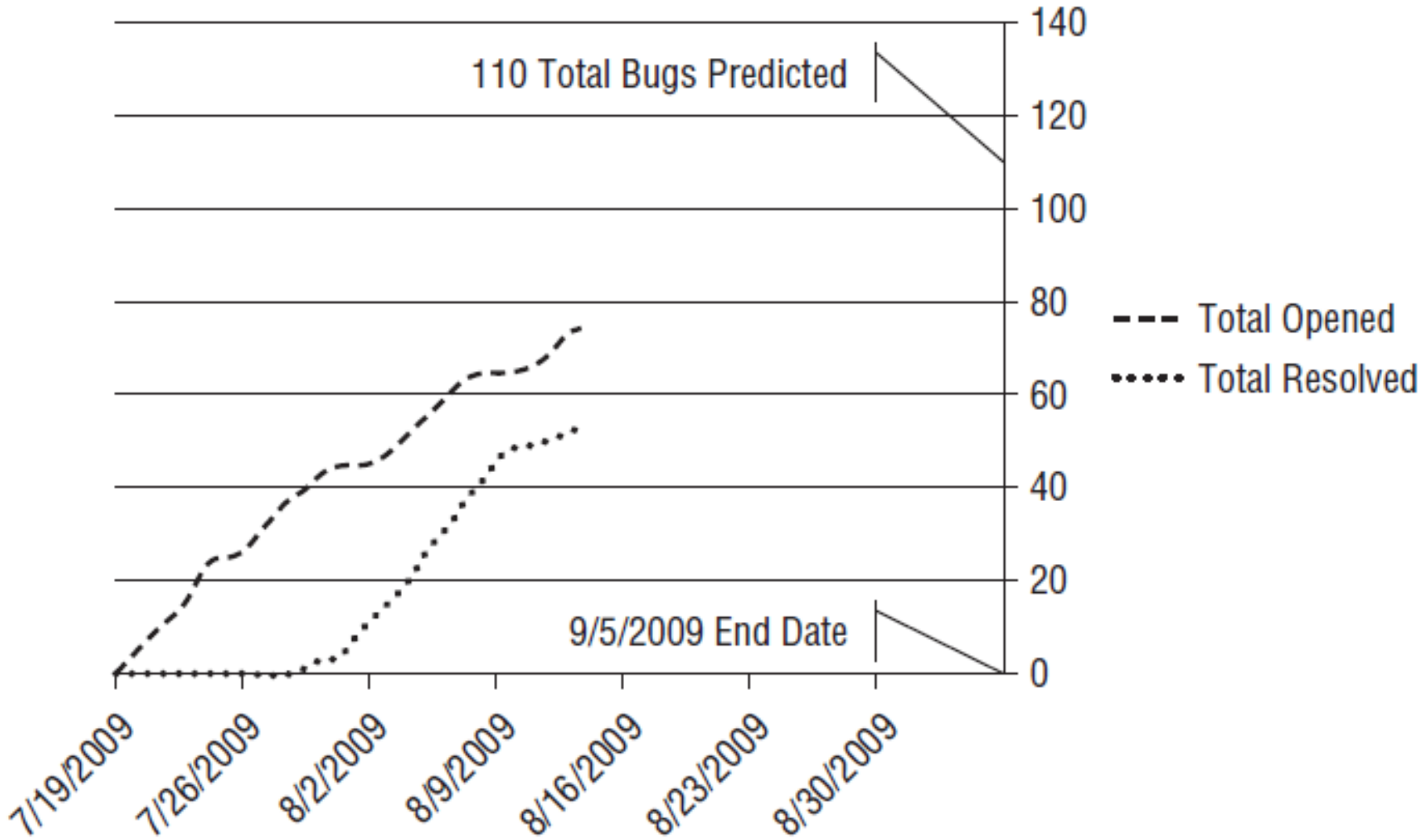
How Defect Removal Proceeds: The Opened/Closed Chart

- The most basic defect analysis chart plots the total number of bugs opened against the total number resolved on a daily basis.
- It's a common chart, and widely referred to as either the *convergence*, *opened/closed*, *found/fixed*, or simply *bug* chart.
- Some charts, offers a general view of the development project's status, taken during the middle of the system test phase.
- This information-rich chart provides answers to a number of questions.
- How close are we to being ready to ship the product?
- In the upcoming chart, you see the predicted number of bugs — created during the estimation work to show where you expect the total number of opened bugs to converge.
- Test projects eventually hit a point at which further testing produces diminishing returns.
- When the Total Opened curve— levels off to an expected number of bugs, testing is winding down, at least for the phase currently underway.

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

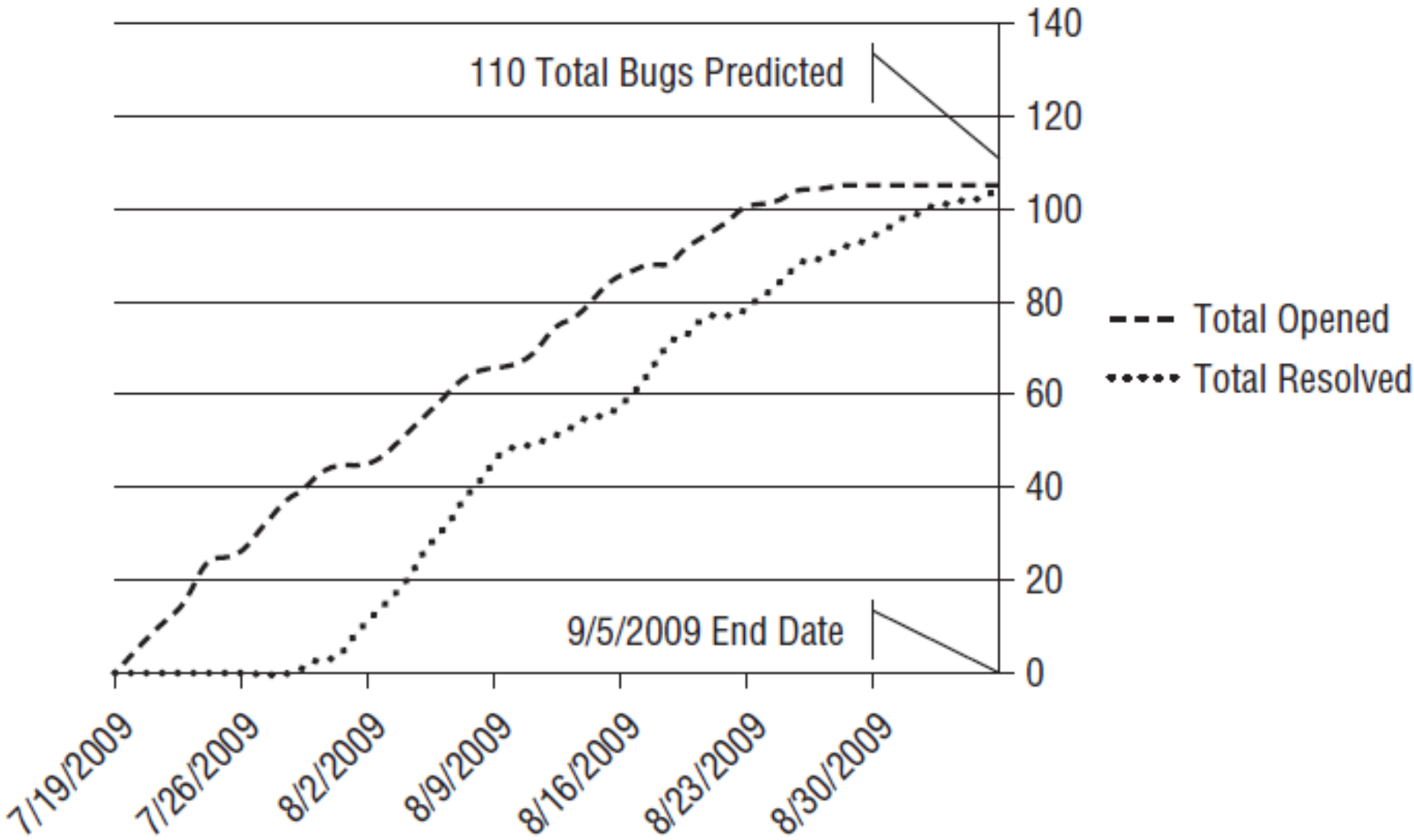
An opened/closed chart for SpeedyWriter



NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

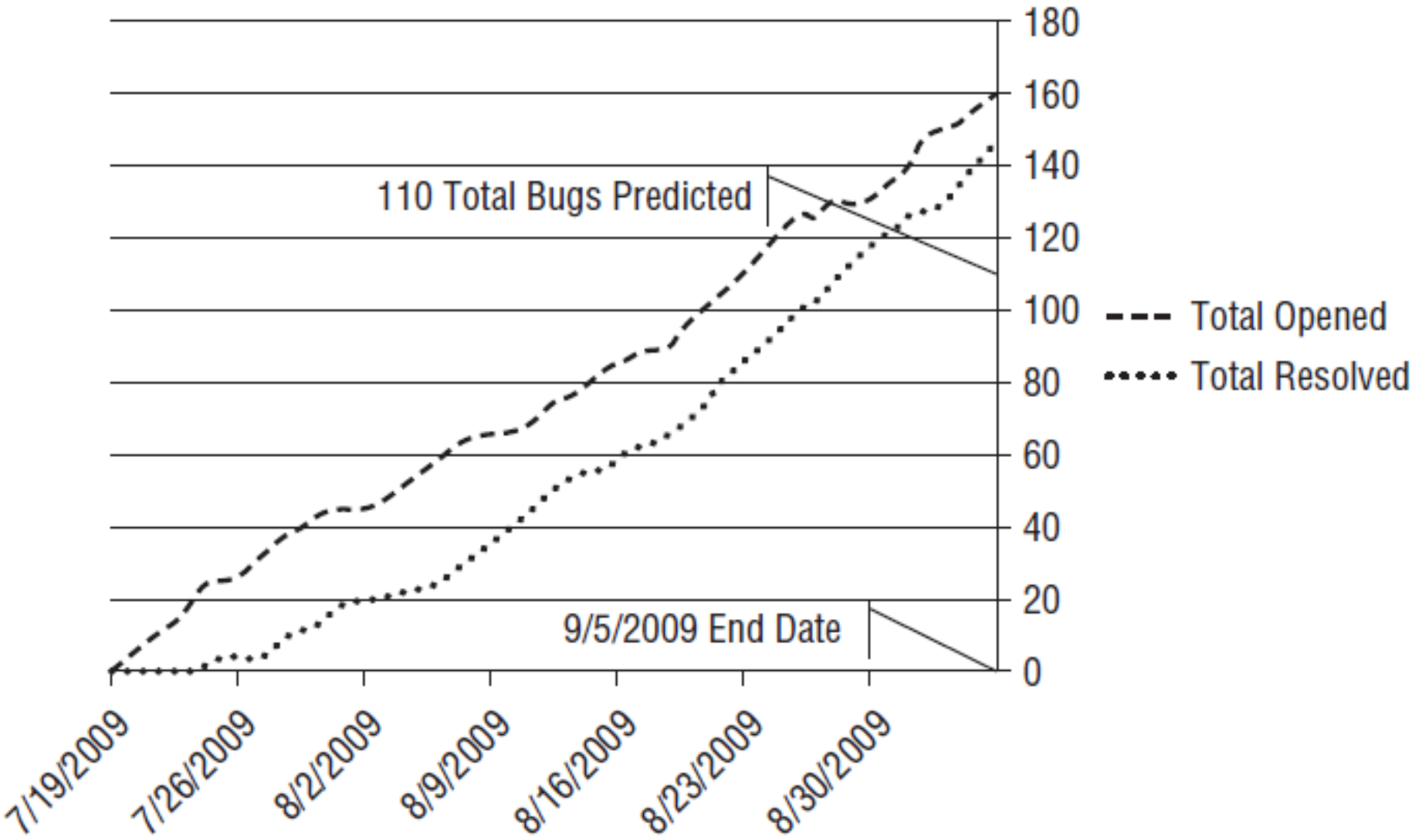
A good-looking opened/closed chart



NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

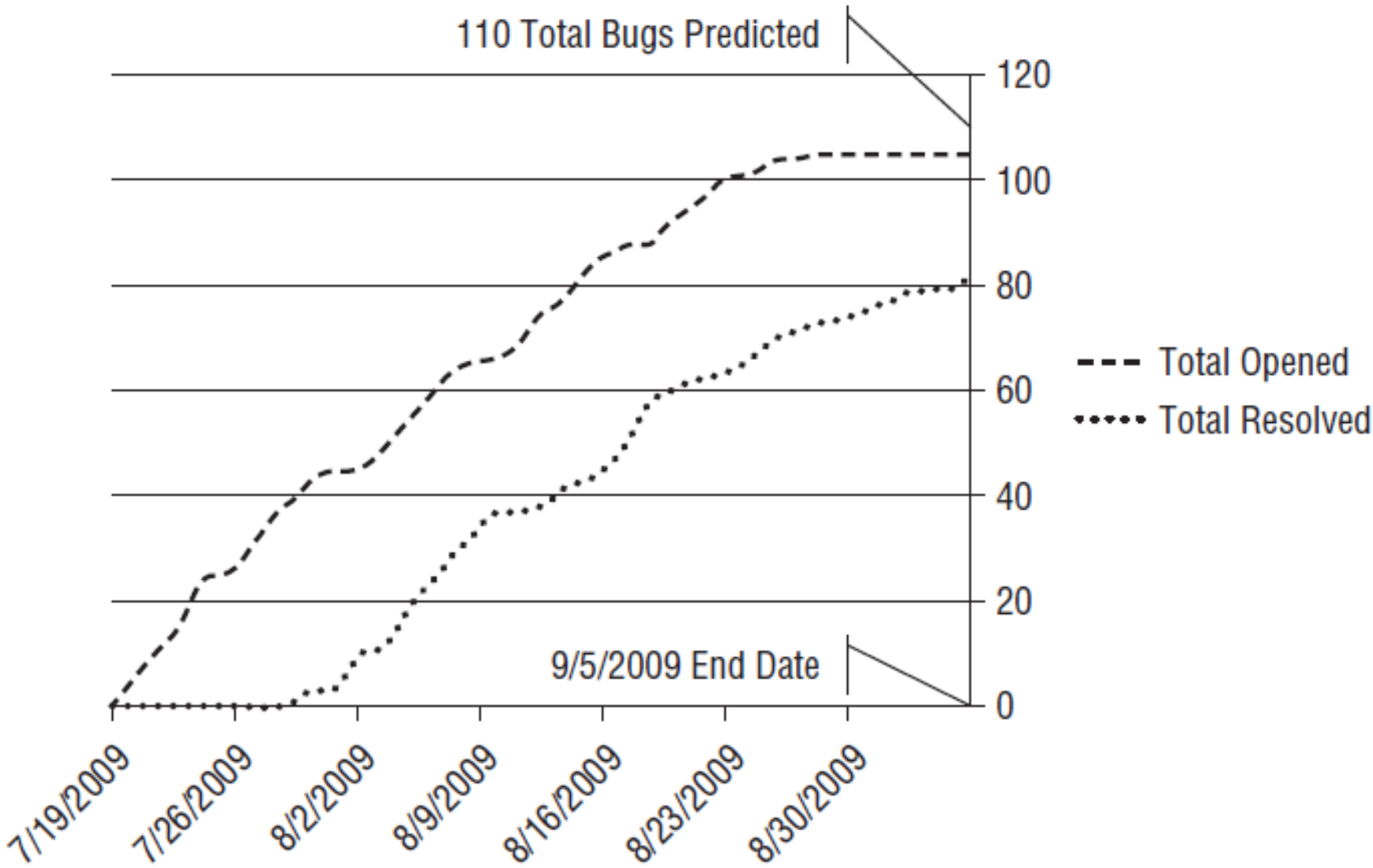
The nightmare of endless bug discovery



NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

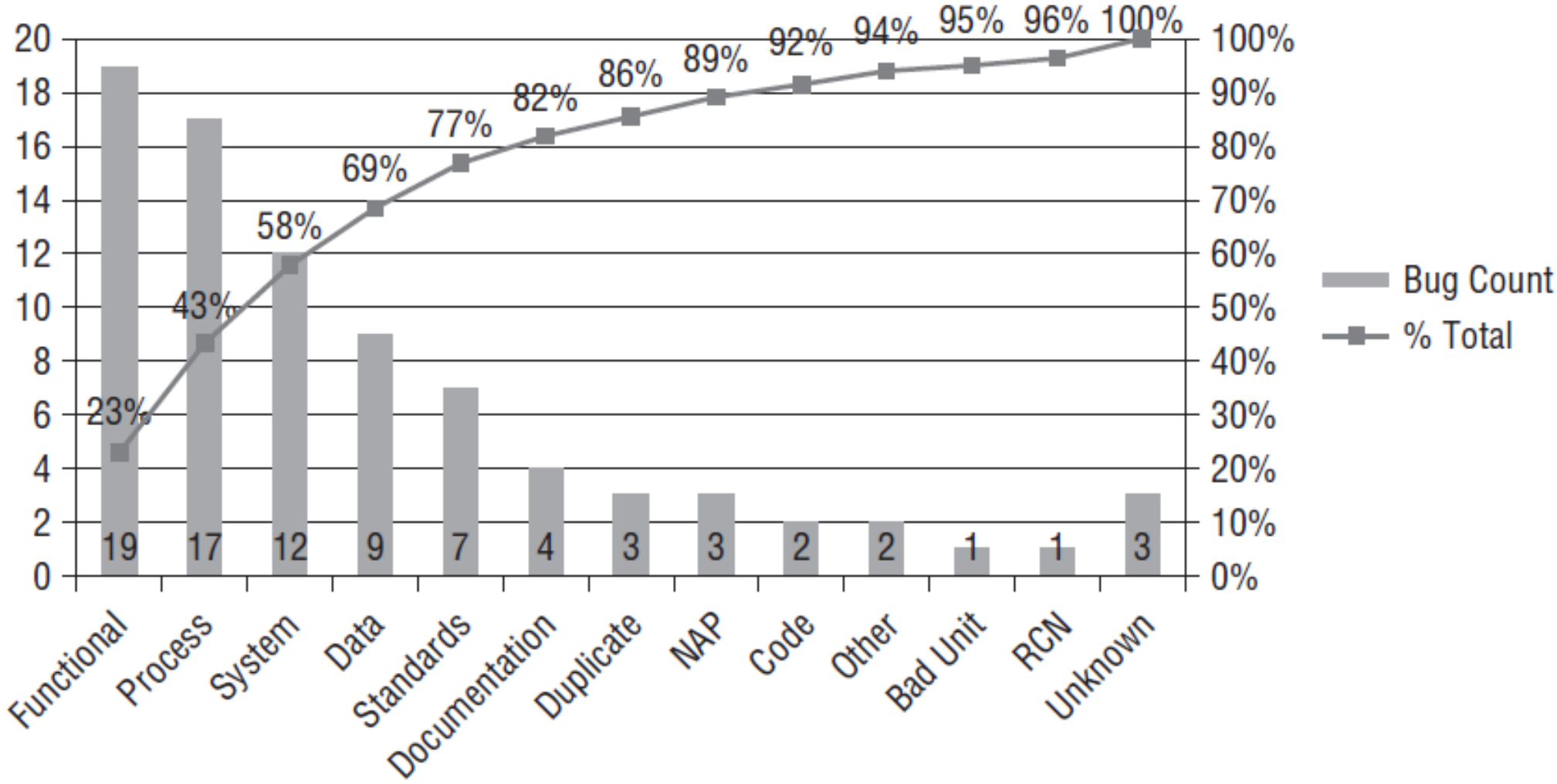
The nightmare of ignored bugs



NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

A root cause breakdown for SpeedyWriter



NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

Which team did a better job? A tale of two test teams

SUBSYSTEM	ACTUAL BUGS	TEST TEAM A	TEST TEAM B
User Interface	100	100	74
Edit Engine	100	100	66
Tools	50	10	20
File	50	8	28
Install/Configuration	50	7	28
Documentation/Packaging	25	7	12
Other	10	2	4
Unknown	10	2	4
N/A	5	2	2
Total	400	238	238

NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Faculty of Computing and Informatics

The IEEE 829 standard's anomaly report template

1. Introduction
   1.1. Document identifier
   1.2. Scope
   1.3. References
2. Details
   2.1. Summary
   2.2. Date anomaly discovered
   2.3. Context
   2.4. Description of anomaly
   2.5. Impact
   2.6. Originator's assessment of urgency
   2.7. Description of the corrective action
   2.8. Status of the anomaly
   2.9. Conclusions and recommendations
3. General
   3.1. Document change procedures and history